Sales Data Warehouse Project

End-to-End ETL Pipeline with Star Schema & Power BI Integration

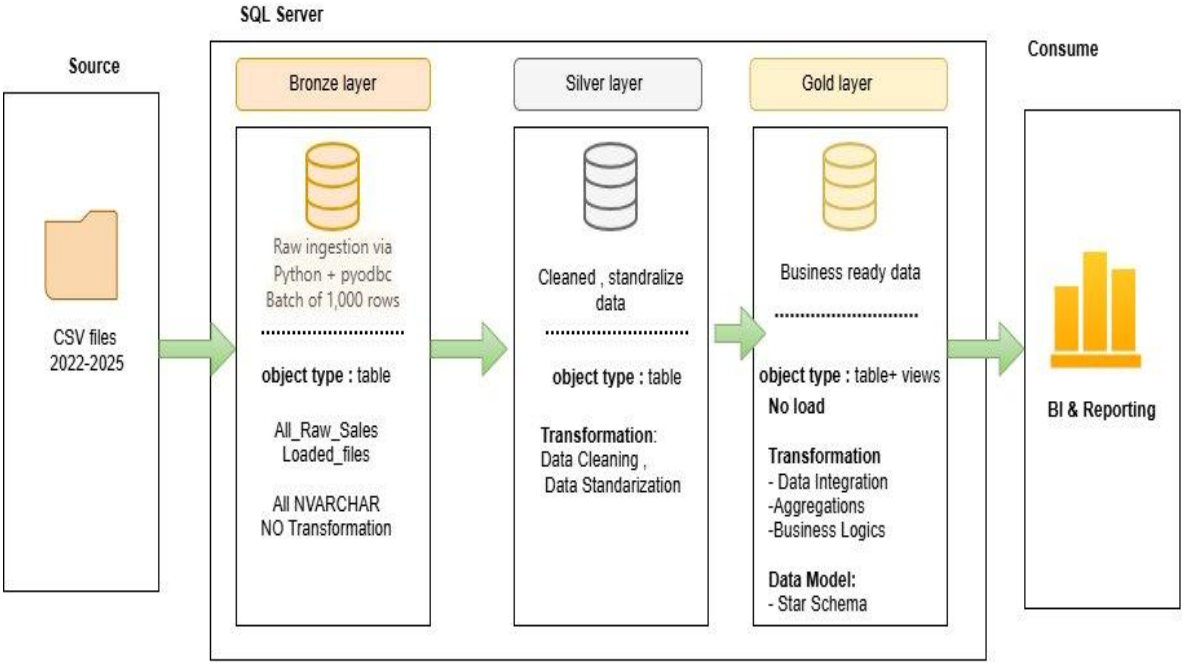
1. Executive Summary

This project delivers a complete, production-ready Data Warehouse for sales analytics. Raw CSV sales data is ingested, cleaned, and transformed through a three-layer Medallion Architecture (Bronze → Silver → Gold), ultimately powering interactive Power BI dashboards.

The solution enables the business to analyze sales performance, customer behavior, product trends, and geographic distribution across a date range spanning 2020 to 2030.

2. Architecture Overview

The project follows the Medallion (Lakehouse) Architecture pattern — a modern data engineering standard that separates raw ingestion from cleansed and business-ready data.



Layer	Schema	Object Type	Purpose
Bronze	bronze	Table	Raw data — all columns NVARCHAR, no transformations
Silver	silver	Table + SP	Cleaned, typed, deduplicated data

Gold	gold	Views + Table	Star Schema — ready for BI consumption

---

3. Layer 1 — Bronze (Raw Ingestion)

Table: bronze.All_Raw_sales

The Bronze layer stores raw data exactly as received from source CSV files. All columns are stored as NVARCHAR to preserve original values without any transformation risk.

- Incremental loading strategy — only new orders (by order_date) are loaded on each run
- Full load on first run when table is empty
- 9,994 total rows successfully ingested across 4 CSV files
- Python automation handles CSV discovery, connection management, and batch insert

Python ETL Script — Overview

The ingestion layer is powered by a Python script (Jupyter Notebook) built using the pyodbc and pandas libraries. The script automates the full Bronze loading process with the following key capabilities:

Feature	Description
Dual Load Strategy	Bulk Load for initial historical files; Incremental Load for future new files — all handled automatically based on file name
Duplicate Prevention	A dedicated loaded_files audit table tracks every processed file. Files already loaded are automatically skipped on re-runs
Batch Processing	Data is inserted in configurable batch sizes (1,000 rows) to avoid memory issues and server timeouts on large files
Date Intelligence	Incremental mode compares incoming Order Dates against the latest date already in the database, loading only genuinely new records
Data Safety	All CSV values are read as strings (dtype=str) and empty values are normalized before insert — no transformation happens at this layer
Audit Trail	Each row carries a source_file column recording which CSV it came from, enabling full lineage tracking

Load Results

File	Rows Loaded	Load Type
Sales 2022.csv	1,993	Bulk
Sales 2023.csv	2,102	Bulk

Sales 2024.csv	2,587	Bulk
Sales 2025.csv	3,312	Bulk
Total	9,994 rows	

4. Layer 2 — Silver (Cleansed & Typed)

Table: silver.AllSales — via Stored Procedure

A dedicated Stored Procedure transforms Bronze data into a clean, strongly-typed Silver table. This layer is the single source of truth for all downstream analytics.

Transformation	Details
Data Type Casting	NVARCHAR → INT, DECIMAL(10,2), DATE as appropriate per column
Date Normalization	6 formats handled: DD/MM/YYYY, MM/DD/YYYY, YYYY-MM-DD, D/M/YYYY, M/D/YYYY, and Excel Serial Numbers
Category Standardization	Segment, Region, and Ship Mode values normalized to consistent casing and naming
Numeric Validation	Sales, Profit, Discount, and Quantity validated for correct ranges and data types
Deduplication	ROW_NUMBER() OVER (PARTITION BY order_id) removes duplicate records, keeping the most recent version

5. Layer 3 — Gold (Star Schema)

Schema: gold — Business-Ready Star Schema

The Gold layer implements a classic Star Schema optimized for Power BI performance. Four dimension objects surround a central fact view.

Object	Type	Key Column	Description
dim_customers	View	Customer_key	Unique customers with segment classification
dim_products	View	product_key	Product catalog with category and sub-category
dim_locations	View	loc_key	Geographic hierarchy: city, state, country, region
dim_dates	Table	date_key (INT)	Date dimension 2020–2030 with day, month, quarter, year attributes

fact_sales	View	order_id	Transactional fact table joining all four dimensions

Surrogate Key Design

Surrogate keys are generated using ROW_NUMBER() OVER (ORDER BY <natural_key>) within each dimension view — ensuring stable sequential integer keys without requiring IDENTITY columns or sequences

---

6. Data Flow Summary

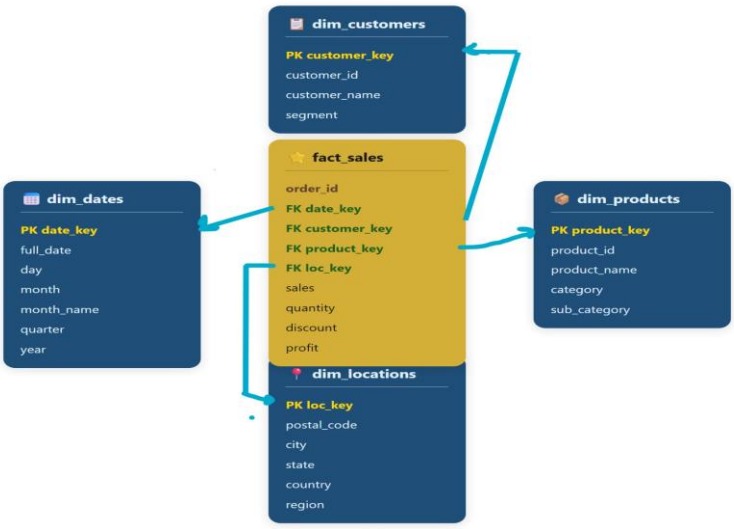
Step	Process	Output
1	Python reads CSV files from source folder	Raw rows ready for insert
2	Audit check: has this file been loaded before?	Skip if yes / proceed if no
3	Batch insert into bronze.All_Raw_sales	9,994 rows in Bronze
4	Execute Silver Stored Procedure	Cleaned rows in silver.AllSales
5	Gold Views auto-refresh on query	Star Schema served to Power BI
6	Power BI connects via SQL Server connector	Interactive dashboards

---

7. Power BI Integration

Relationship Model

- fact_sales[date_key] → dim_dates[date_key]
- fact_sales[customer_key] → dim_customers[Customer_key]
- fact_sales[Product_key] → dim_products[product_key]
- fact_sales[Location_key] → dim_locations[loc_key]



8. Data Quality & Validation

Before finalizing each layer, the following checks were applied:

Check	Method	Layer
No duplicate orders	ROW_NUMBER() PARTITION BY order_id	Silver
Valid date formats	TRY_CONVERT with fallback handling for 6 formats	Silver
No null surrogate keys	WHERE clause filters on all dimension views	Gold
Referential integrity	LEFT JOINs in fact_sales expose unmatched keys as NULL	Gold
Date dimension coverage	dim_dates spans 2020–2030 (3,287+ rows)	Gold

9. Security & Access Control

To enforce the principle of least privilege, a Role-Based Access Control (RBAC) model was implemented using SQL Server Logins, Database Users, and a custom Database Role.

Implementation Steps

Step	Command	Scope	Description
1	CREATE LOGIN	master	Created server-level logins with passwords for each user
2	CREATE USER	SalesDWHP	Mapped each login to a database user in SalesDWHP
3	CREATE ROLE	SalesDWHP	Created a custom role named SeniorDataAnalyst
4	GRANT SELECT	SalesDWHP	Granted SELECT on all 5 Gold Layer views to the role
5	ALTER ROLE ADD MEMBER	SalesDWHP	Added the named user as a member of the role

Users & Permissions

User	Role Member	Gold Views Access	Silver / Bronze Access
YourName	SeniorDataAnalyst	Yes	No

Other_User	Not a member	No	No

This design ensures that analysts can query the Gold Layer for reporting, while raw and intermediate data in the Bronze and Silver layers remains protected.

---

10. Project Deliverables

File	Description
load.ipynb	Python ETL Notebook — Bronze ingestion with dual load strategy
SilverStoredProcedure.sql	T-SQL Stored Procedure for Bronze → Silver transformation and cleansing
GoldLayer.sql	Gold Layer DDL: dim_customers, dim_products, dim_locations, fact_sales Views + dim_dates Table

---

11. Conclusion

The Sales Data Warehouse project successfully delivers a scalable, maintainable, and analytically powerful solution. The three-layer Medallion Architecture ensures data quality at each stage, while the Star Schema in the Gold layer provides optimal performance for Power BI reporting.

The pipeline is fully re-runnable: each layer can be refreshed independently without data integrity issues, and the incremental loading strategy in the Bronze layer minimizes processing time on subsequent runs.

Role-Based Access Control ensures that only authorized users can access the curated Gold Layer, while all raw and intermediate data remains protected.